Luca De Simone 1592157

VFAT:

No journaling; meta in FAT; no hardlink

nur dir entries, keine inodes.

File names up to 255 chars (unicode)

FAT[cluster_number] = cluster > fixed size

ext3:

dir entries die auf inodes zeigen ; scales well; permissions

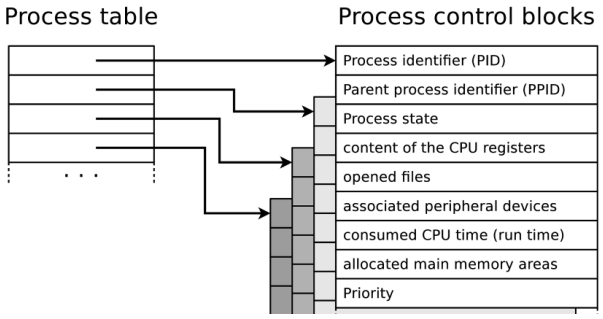
4kb pro address cluster

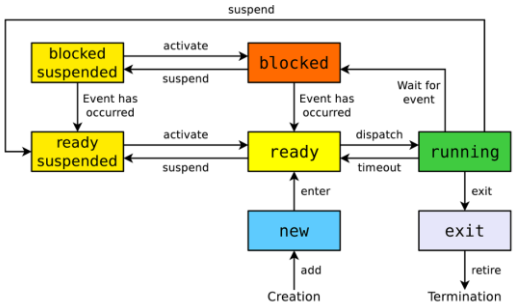
12 pointer pro block -> 4 * 12 = 48kb,

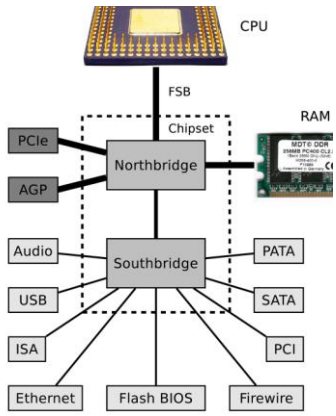
alles darüber hinaus mit indirect pointers:

pointer auf einen block mit 12 pointer: ...

- For **Linux** systems:
  - Priorities between `-20` and `+19` are available
  - `-20` is the highest priority and `+19` is the lowest priority.
  - The default priority is `0`
  - Normal users can assign priorities from `0` to `19`
  - The super user (`root`) can assign negative values too

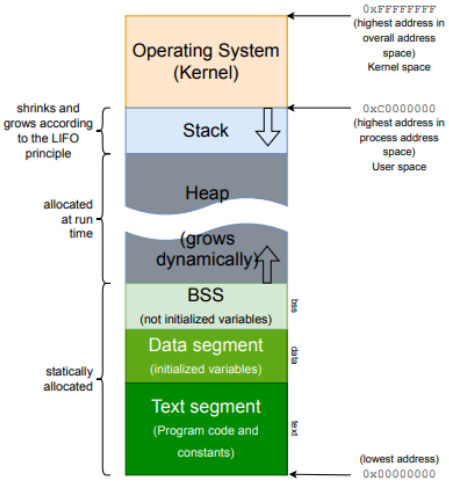






FAT File System Structure

Region 1	Region 2		Region 3	Region 4
Reserved blocks	FAT1	FAT2	Root directory	Data



Non-Preemptive-Sched.: process will either run until end or voluntarily yield

Preemptive-Sched.: System can reeasign CPU

Priority Driven: Just prio >; FCFS:FIFO; LCFS:LIFO; LCFS-PR:Stack;

EDF:Earliest Deadline First; SJF:Shortest First;

	cooperative	preemptive	Fair	Priortiy based
Priority-driven scheduling	X	X	no	yes
First Come First Served = FIFO	X		yes	no
Last Come First Served	X	X	no	no
Round Robin		X	yes	no
Shortest/Longest Job First	X		no	no
Shortest Remaining Time First		X	no	no
Longest Remaining Time First		X	no	no
Highest Response Ratio Next	X		yes	no
Earliest Deadline First	X	X	yes	no
Static multilevel scheduling		X	no	yes (static)
Multilevel feedback scheduling		X	yes	yes (dynamic)
Earliest Eligible Virtual Deadline First		X	yes	yes

Hardware Interrupts: Await phys. Signal, timer tick

Exceptions: 0 div, throw …, segfault, s.ovfl., …

Traps: Software interrupt to cause change of op. Mode

IRQ: Interupt Request

Seq. Interrupt Req.: One Interrupt after the other

+: Simpler -: No prio > bad reaction

Nested Interrupt Req.: Interrupt Handlers can be Interrupted (when higher p)

-: complicated, Interrupts may starve / delay because prio

ISR: Interrupt service routine: Int. > Save prog. > Routine > Load prog. > end

„A system call is a function call in the operating system that triggers a switch from user mode to kernel mode“

IPC: Inter Process Communication: Comm. (msg) & Cooperation (shared)

- Files, Signals/Flags, Shared mem, msg q, pipes, sockets

	Shared Memory	Message Queues	(anon./named) Pipes	Socke
Scheme	Memory-based	Message-based	Stream-based	Message-
Bidirectional	yes	no	no	yes
Platform independent	no	no	no	yes
Processes relation required	no	no	for anon. pipes	no
Common address space required	yes	yes	yes	no
Bound to a process	no	on	yes	yes
Automatic synchronization	no	yes	yes	yes

A deadlock situation can arise if these conditions are all fulfilled – **Mutual exclusion** ∗ At least one resource is either occupied by exactly one process or is available ⟹ non-sharable resource – **Hold and wait** ∗ A process, which currently occupies at least one resource, requests additional resources which are being held by another process – **No preemption** ∗ Resources occupied by a process cannot be deallocated by the OS but only be released by the holding process voluntarily – **Circular wait** ∗ A cyclic chain of processes exists ∗ Each process requests a resource that the next process in the chain occupies.

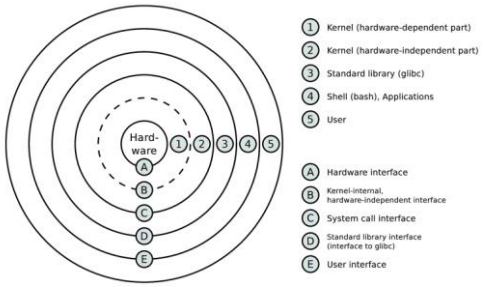
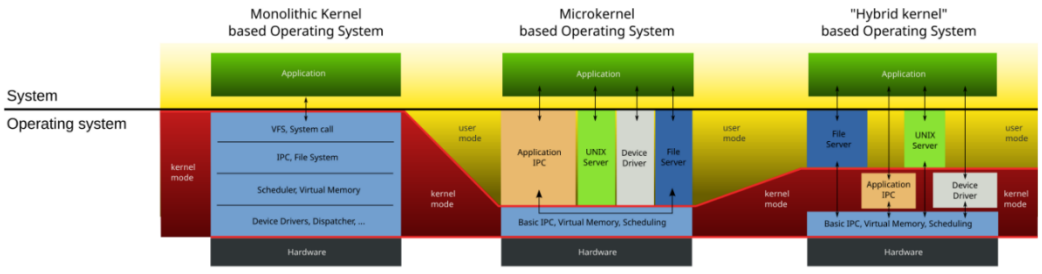
Graph:: Process: Circle, Resource: Rectangle

Current allocation Matrix: What we have currently

Request Matrix: What each process needs.

Semaphore: P(), V(), =counter

P: if sema > 0: sema --; else block process

V: sema++; if process waiting: wake one up

Mutex: Bin. Sema. > „Besitzer“ > only one controller